

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Внешние методы сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

I. Метод естественного слияния

1. Постановка задачи

Отсортировать массив методом естественного слияния

2. Анализ задачи

Деление на серии

Попарное слияние серий с одновременной сортировкой

Повторение шагов (1-2) до получения единственной серии, то есть полностью отсортированных данных

3. Код

```
#include <iostream>
using namespace std;
```

```
void printArray(int arr[], int size, string msg = "") {
    cout << msg;
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int findNaturalRuns(int arr[], int size, int boundaries[]) {
    int count = 0;
    int start = 0;

    for (int i = 1; i <= size; i++) {
        if (i == size || arr[i] < arr[i - 1]) {
            boundaries[count++] = i;
            start = i;
        }
    }

    return count;
}

void merge(int arr[], int temp[], int left, int mid, int right) {
    int i = left;
    int j = mid;
    int k = left;

    while (i < mid && j < right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        }
        else {
            temp[k++] = arr[j++];
        }
    }

    while (i < mid) {
        temp[k++] = arr[i++];
    }
    while (j < right) {
        temp[k++] = arr[j++];
    }
}
```

```

    for (int idx = left; idx < right; idx++) {
        arr[idx] = temp[idx];
    }
}

void naturalMergeSort(int arr[], int size) {
    int* temp = new int[size];
    int boundaries[100];
    int step = 1;

    printArray(arr, size, "Исходный массив: ");

    while (true) {
        int numRuns = findNaturalRuns(arr, size, boundaries);

        cout << "\nШаг " << step++ << endl;
        cout << "Найдено серий: " << numRuns << endl;
        cout << "Границы серий: ";
        for (int i = 0; i < numRuns; i++) {
            cout << boundaries[i] << " ";
        }
        cout << endl;

        if (numRuns == 1) {
            cout << "Массив отсортирован!" << endl;
            break;
        }

        cout << "Слияние серий:" << endl;
        int i = 0;
        while (i < numRuns - 1) {
            int left = (i == 0) ? 0 : boundaries[i - 1];
            int mid = boundaries[i];
            int right = boundaries[i + 1];

            cout << " Сливаем [" << left << "-" << mid - 1 << "] и ["
                << mid << "-" << right - 1 << "]" << endl;

            merge(arr, temp, left, mid, right);

            printArray(arr, size, " После слияния: ");

            i += 2;
        }

        if (numRuns % 2 == 1) {
            cout << " Последняя серия остается без изменений" << endl;
        }
    }

    printArray(arr, size, "Отсортированный: ");

    delete[] temp;
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 41, 9, 15, 24, 2, 1, 12, 7 };
    int n = sizeof(arr) / sizeof(arr[0]);

    naturalMergeSort(arr, n);
}

```

```
    return 0;  
}
```

4. Результат выполнения программы

```
Исходный массив: 41 9 15 24 2 1 12 7  
  
Шаг 1  
Найдено серий: 5  
Границы серий: 1 4 5 7 8  
Слияние серий:  
    Сливаем [0-0] и [1-3]  
    После слияния: 9 15 24 41 2 1 12 7  
    Сливаем [4-4] и [5-6]  
    После слияния: 9 15 24 41 1 2 12 7  
    Последняя серия остается без изменений  
  
Шаг 2  
Найдено серий: 3  
Границы серий: 4 7 8  
Слияние серий:  
    Сливаем [0-3] и [4-6]  
    После слияния: 1 2 9 12 15 24 41 7  
    Последняя серия остается без изменений  
  
Шаг 3  
Найдено серий: 2  
Границы серий: 7 8  
Слияние серий:  
    Сливаем [0-6] и [7-7]  
    После слияния: 1 2 7 9 12 15 24 41  
  
Шаг 4  
Найдено серий: 1  
Границы серий: 8  
Массив отсортирован!  
Отсортированный: 1 2 7 9 12 15 24 41
```

II. Сбалансированное двухфазное слияние

1. Постановка задачи

Отсортировать массив методом двухфазного слияния

2. Анализ задачи

Целочисленное деление множества на две части и запись элементов множества в два подмножества

Если количество элементов множества нечетное, то дополнительный элемент записывается в левое подмножество

Повторное целочисленное деление множества. Деление происходит до тех пор, пока не останется 2 элемента

Внутри каждого подмножества элементы сортируются, например, по возрастанию и сливаются попарно

Сортировка и слияние происходит до получения отсортированного множества

Получение отсортированного множества

3. Код

```
#include <iostream>
using namespace std;

// Функция вывода массива
void printArray(int arr[], int size, string msg = "") {
    cout << msg;
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

// Функция слияния двух отсортированных частей
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Создаем временные массивы
    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
}
```

```

        j++;
    }
    k++;
}

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

delete[] L;
delete[] R;
}

// Основная функция сортировки
void balancedMergeSort(int arr[], int left, int right, int& step, int size) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        cout << "\nШаг " << step++ << "] РАЗДЕЛЕНИЕ диапазона [" << left << "-" << right << "]" << endl;
        cout << "    Левая часть: [" << left << "-" << mid << "]" << endl;
        cout << "    Правая часть: [" << mid + 1 << "-" << right << "]" << endl;

        // Рекурсивный вызов для левой части
        balancedMergeSort(arr, left, mid, step, size);

        // Рекурсивный вызов для правой части
        balancedMergeSort(arr, mid + 1, right, step, size);

        cout << "\nШаг " << step++ << "] СЛИЯНИЕ: [" << left << "-" << mid << "]" + [" << mid + 1 << "-" << right
        << "]" << endl;

        // Выводим подмассивы, которые сливаем
        cout << "    Левый подмассив: ";
        for (int i = left; i <= mid; i++) cout << arr[i] << " ";
        cout << endl;

        cout << "    Правый подмассив: ";
        for (int i = mid + 1; i <= right; i++) cout << arr[i] << " ";
        cout << endl;

        merge(arr, left, mid, right);

        // Выводим результат слияния (весь массив для наглядности)
        cout << "    Результат слияния: ";
        printArray(arr, size);
    }
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 6, 2, 3, 7, 1, 9, 5, 8, 4 };
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Исходный массив: ";

```

```

    printArray(arr, n);

    int step = 1;
    balancedMergeSort(arr, 0, n - 1, step, n);

    cout << "Итоговый массив: ";
    printArray(arr, n);

    return 0;
}

```

4. Результат выполнения программы

Исходный массив: 6 2 3 7 1 9 5 8 4

[Шаг 1] РАЗДЕЛЕНИЕ диапазона [0-8]
 Левая часть: [0-4]
 Правая часть: [5-8]

[Шаг 2] РАЗДЕЛЕНИЕ диапазона [0-4]
 Левая часть: [0-2]
 Правая часть: [3-4]

[Шаг 3] РАЗДЕЛЕНИЕ диапазона [0-2]
 Левая часть: [0-1]
 Правая часть: [2-2]

[Шаг 4] РАЗДЕЛЕНИЕ диапазона [0-1]
 Левая часть: [0-0]
 Правая часть: [1-1]

[Шаг 5] СЛИЯНИЕ: [0-0] + [1-1]
 Левый подмассив: 6
 Правый подмассив: 2
 Результат слияния: 2 6 3 7 1 9 5 8 4

[Шаг 6] СЛИЯНИЕ: [0-1] + [2-2]
 Левый подмассив: 2 6
 Правый подмассив: 3
 Результат слияния: 2 3 6 7 1 9 5 8 4

[Шаг 7] РАЗДЕЛЕНИЕ диапазона [3-4]
 Левая часть: [3-3]
 Правая часть: [4-4]

[Шаг 8] СЛИЯНИЕ: [3-3] + [4-4]
 Левый подмассив: 7
 Правый подмассив: 1
 Результат слияния: 2 3 6 1 7 9 5 8 4

[Шаг 9] СЛИЯНИЕ: [0-2] + [3-4]
 Левый подмассив: 2 3 6
 Правый подмассив: 1 7
 Результат слияния: 1 2 3 6 7 9 5 8 4

[Шаг 10] РАЗДЕЛЕНИЕ диапазона [5-8]
 Левая часть: [5-6]
 Правая часть: [7-8]

[Шаг 11] РАЗДЕЛЕНИЕ диапазона [5-6]
 Левая часть: [5-5]
 Правая часть: [6-6]

[Шаг 12] СЛИЯНИЕ: [5-5] + [6-6]
 Левый подмассив: 9
 Правый подмассив: 5
 Результат слияния: 1 2 3 6 7 5 9 8 4

[Шаг 13] РАЗДЕЛЕНИЕ диапазона [7-8]
 Левая часть: [7-7]
 Правая часть: [8-8]

[Шаг 14] СЛИЯНИЕ: [7-7] + [8-8]
 Левый подмассив: 8
 Правый подмассив: 4
 Результат слияния: 1 2 3 6 7 5 9 4 8

[Шаг 15] СЛИЯНИЕ: [5-6] + [7-8]
 Левый подмассив: 5 9
 Правый подмассив: 4 8
 Результат слияния: 1 2 3 6 7 4 5 8 9

[Шаг 16] СЛИЯНИЕ: [0-4] + [5-8]
 Левый подмассив: 1 2 3 6 7
 Правый подмассив: 4 5 8 9
 Результат слияния: 1 2 3 4 5 6 7 8 9
 Итоговый массив: 1 2 3 4 5 6 7 8 9

III. Многофазная сортировка

1. Постановка задачи

Отсортировать массив многофазной сортировкой

2. Анализ задачи

1. Начальное распределение серий

Исходные данные разбиваются на серии и распределяются по нескольким файлам.

Количество серий в каждом файле должно соответствовать определённой последовательности (например, числам Фибоначчи).

2. Слияние серий

Серии из нескольких файлов сливаются в один, при этом на каждом этапе количество серий уменьшается.

Фиктивные серии используются для балансировки процесса.

3. Повторение процесса

Процесс слияния повторяется до тех пор, пока не останется одна полностью отсортированная серия.

3. Код

```
#include <iostream>
using namespace std;

const int MAX_SIZE = 100;

void printArray(int arr[], int size, string msg = "") {
    cout << msg;
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int mergeArrays(int a[], int sizeA, int b[], int sizeB, int result[]) {
    int i = 0, j = 0, k = 0;

    while (i < sizeA && j < sizeB) {
        if (a[i] <= b[j]) {
            result[k++] = a[i++];
        }
        else {
            result[k++] = b[j++];
        }
    }

    while (i < sizeA) result[k++] = a[i++];
    while (j < sizeB) result[k++] = b[j++];

    return k;
}

void multiPhaseSort(int arr[], int size) {
    int tape1[MAX_SIZE], tape2[MAX_SIZE], tape3[MAX_SIZE];
```



```

int size1 = 0, size2 = 0, size3 = 0;
int step = 1;

printArray(arr, size, "Исходный: ");

cout << "\nРаспределение" << endl;

for (int i = 0; i < size; i++) {
    if (i % 2 == 0) {
        tape1[size1++] = arr[i];
    }
    else {
        tape2[size2++] = arr[i];
    }
}

cout << "Лента 1: ";
printArray(tape1, size1);
cout << "Лента 2: ";
printArray(tape2, size2);

for (int i = 0; i < size1 - 1; i++) {
    for (int j = i + 1; j < size1; j++) {
        if (tape1[i] > tape1[j]) {
            int temp = tape1[i];
            tape1[i] = tape1[j];
            tape1[j] = temp;
        }
    }
}

for (int i = 0; i < size2 - 1; i++) {
    for (int j = i + 1; j < size2; j++) {
        if (tape2[i] > tape2[j]) {
            int temp = tape2[i];
            tape2[i] = tape2[j];
            tape2[j] = temp;
        }
    }
}

cout << "\nПосле сортировки лент:" << endl;
cout << "Лента 1: ";
printArray(tape1, size1);
cout << "Лента 2: ";
printArray(tape2, size2);

cout << "\nСлияние" << endl;

int mergedSize = mergeArrays(tape1, size1, tape2, size2, tape3);

cout << "Результат слияния (Лента 3): ";
printArray(tape3, mergedSize);

for (int i = 0; i < mergedSize; i++) {
    arr[i] = tape3[i];
}

printArray(arr, size, "Отсортирован: ");
}

int main() {
    setlocale(LC_ALL, "ru");

```

```
int arr[] = { 15, 3, 8, 1, 12, 7, 4, 10, 5, 9, 2, 6, 11, 13, 14 };  
int n = sizeof(arr) / sizeof(arr[0]);  
  
multiPhaseSort(arr, n);  
  
return 0;  
}
```

4. Результат выполнения программы

```
Исходный: 15 3 8 1 12 7 4 10 5 9 2 6 11 13 14  
  
Распределение  
Лента 1: 15 8 12 4 5 2 11 14  
Лента 2: 3 1 7 10 9 6 13  
  
После сортировки лент:  
Лента 1: 2 4 5 8 11 12 14 15  
Лента 2: 1 3 6 7 9 10 13  
  
Слияние  
Результат слияния (Лента 3): 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15  
Отсортирован: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```